Practical Lab Report
Access Control Models

DAC • MAC • RBAC • ABAC

Course:	Cybersecurity
Environment:	Arch Linux / Omarchy terminal
System account:	`ilyass`
Lab date:	10 November 2026
Source:	*Practical Lab: Access Control Models*

Evidence-backed report. Observed commands and outputs are distinguished from proposed corrections and theoretical answers.

Contents

1 Introduction and objectives **2**

2 DAC: Discretionary Access Control **2**

2.1 Guided experiment: permission bits 2

2.2 Challenge 1: Confidential Documents 2

3 MAC: Mandatory Access Control **4**

3.1 Concepts and SELinux exploration 4

3.2 Challenge 2: Military Information System 5

4 RBAC: Role-Based Access Control **6**

4.1 Guided experiment: users, roles, and directories 6

4.2 Challenge 3: Enterprise Access Management 7

5 ABAC: Attribute-Based Access Control **8**

5.1 Challenge 4: Banking Access Policy 8

6 Summary and verification status **8**

1 Introduction and objectives

Access control determines who may access a resource and which actions they may perform. This lab explores four models: discretionary access control (DAC), mandatory access control (MAC), role-based access control (RBAC), and attribute-based access control (ABAC). The objectives are to configure Linux permissions, reason about mandatory classification, simulate roles with groups and ACLs, and evaluate context-dependent authorization rules.

Model	Authorization basis
DAC	Owner, group, and file permissions
MAC	Centrally enforced security classifications and policies
RBAC	Organizational roles and their assigned privileges
ABAC	Subject, object, action, and environment attributes

2 DAC: Discretionary Access Control

2.1 Guided experiment: permission bits

A local file named `report.txt` was inspected with `ls -l`. The screenshot shows owner permission `rwx`, group permission `r-`, and no permissions for others: `740`. Its size was zero bytes. For regular files, execute permission does not simply mean "can open"; it permits execution when the file is a valid executable or script.

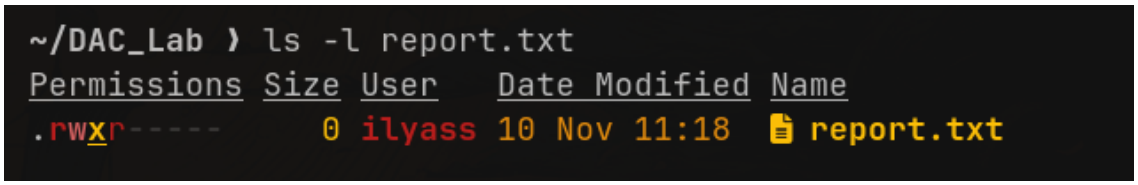


Figure 1: Guided DAC experiment: the displayed `report.txt` permissions correspond to mode `740`.

2.2 Challenge 1: Confidential Documents

Policy: the owner reads and modifies financial documents, accounting group members read them, and all other users receive no access.

Tasks 1–4: creation, group, ownership, and permissions. The following commands implement the policy:

```
sudo mkdir -p /company/finance
sudo touch /company/finance/report.txt
sudo groupadd accounting
sudo chown ilyass:accounting /company/finance
sudo chown ilyass:accounting /company/finance/report.txt
sudo chmod 750 /company/finance
sudo chmod 640 /company/finance/report.txt
```

The screenshots show that `accounting` already existed at one stage, was deleted and then recreated successfully. The stray standalone `-` caused a harmless shell `command not found` error; it was not part of the access-control configuration.

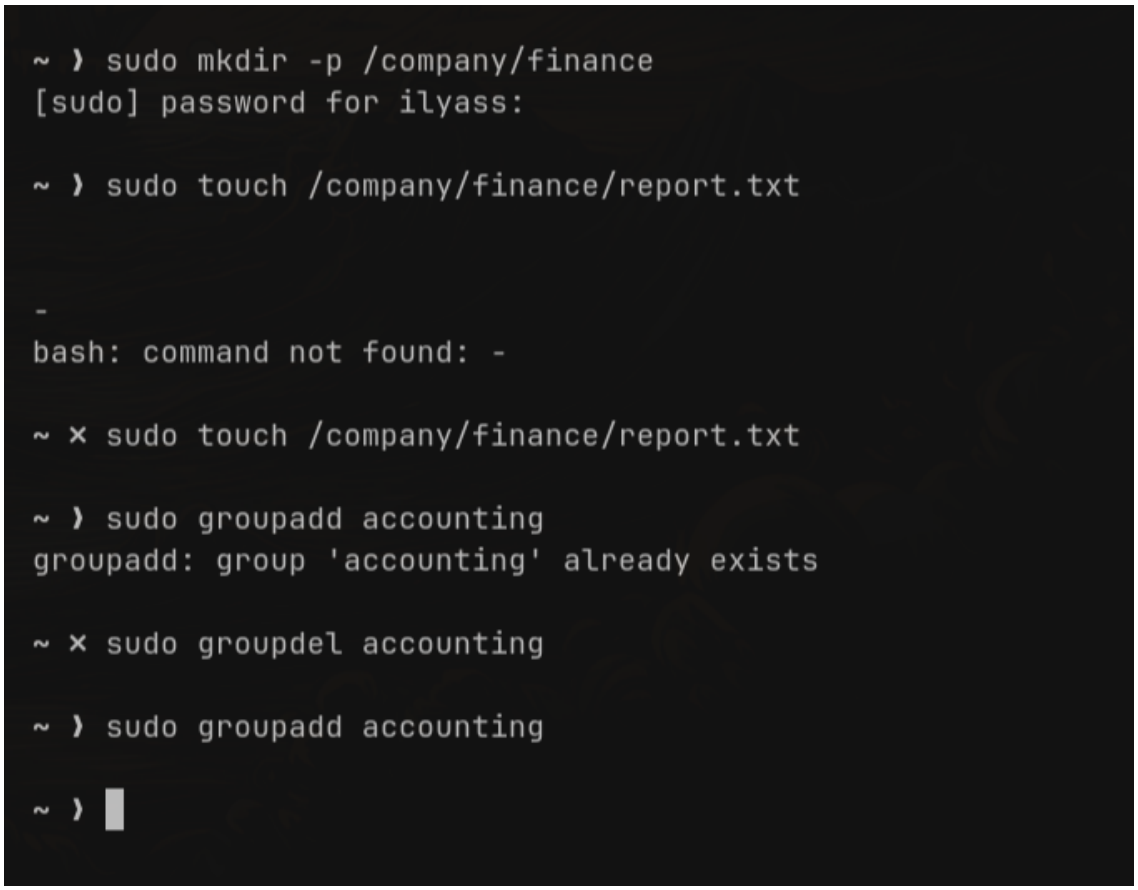


Figure 2: Challenge 1: directory/file creation and accounting-group creation; initial duplicate-group and shell errors are visible.

Task 5: verify final configuration. The submitted screenshot includes both `ls -ld /company/finance` and `ls -l /company/finance/report.txt`. The displayed directory permission is `drwxr-x--` (750), and the document permission is `-rw-r---` (640). The screenshot's configured owner is `ilyass`. Its custom terminal output does not expose a group column, so group ownership is supported by the displayed `chown` commands, not independently demonstrated by that listing.

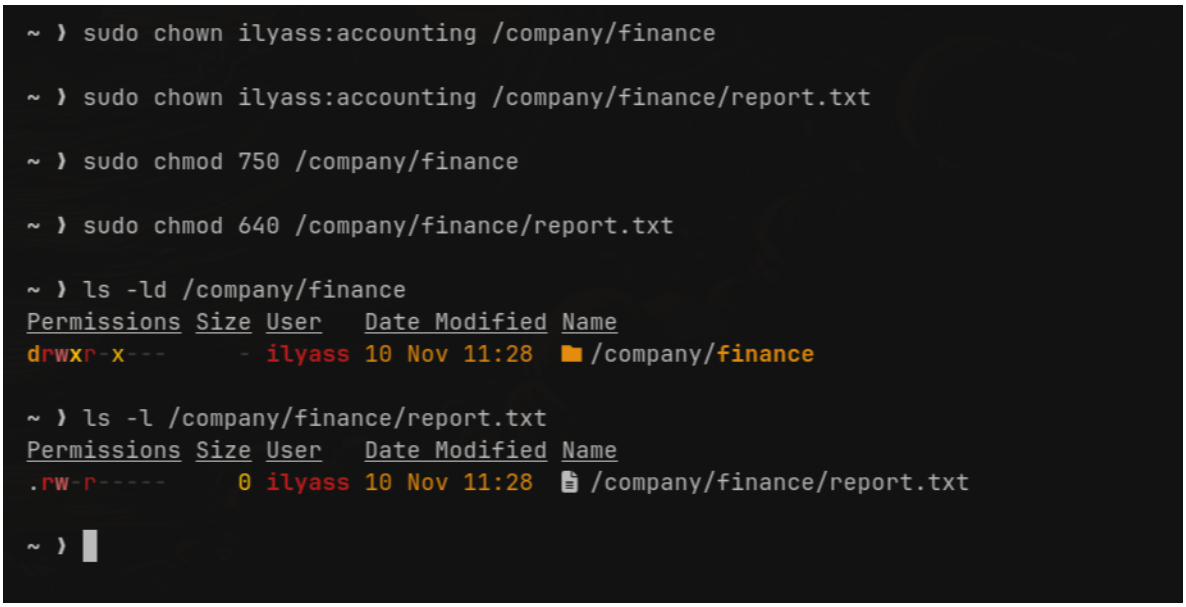


Figure 3: Challenge 1: ownership commands, chmod settings, and verification of directory mode 750 and file mode 640.

Object	Mode	Interpretation
`/company/finance`	750	Owner `rwx`; accounting `r-x`; others none
`report.txt`	640	Owner `rw-`; accounting `r--`; others none

Task 6: directory 750 but file 600. Answer. No, an ordinary accounting-group member cannot read the file. Permission `x` on a directory allows traversal, but reading file contents also requires `r` on the file. Mode `600` grants file read/write only to the owner. Parent-directory traversal permissions are also required. This answer assumes no additional ACL authorization or special privileged identity.

3 MAC: Mandatory Access Control

3.1 Concepts and SELinux exploration

Under MAC, a centrally enforced policy determines access, rather than the resource owner alone. Bell–LaPadula prioritizes confidentiality through *no read up* and *no write down*. The lab also asks for SELinux status (`sestatus`) and contexts (`ls -Z`).

The supplied screenshot shows a terminal run of `ls -Z`, including question marks in the security-context column. This does *not* demonstrate an active, enforcing SELinux policy; there is no screenshot of the `sestatus` result. The status must be verified separately.

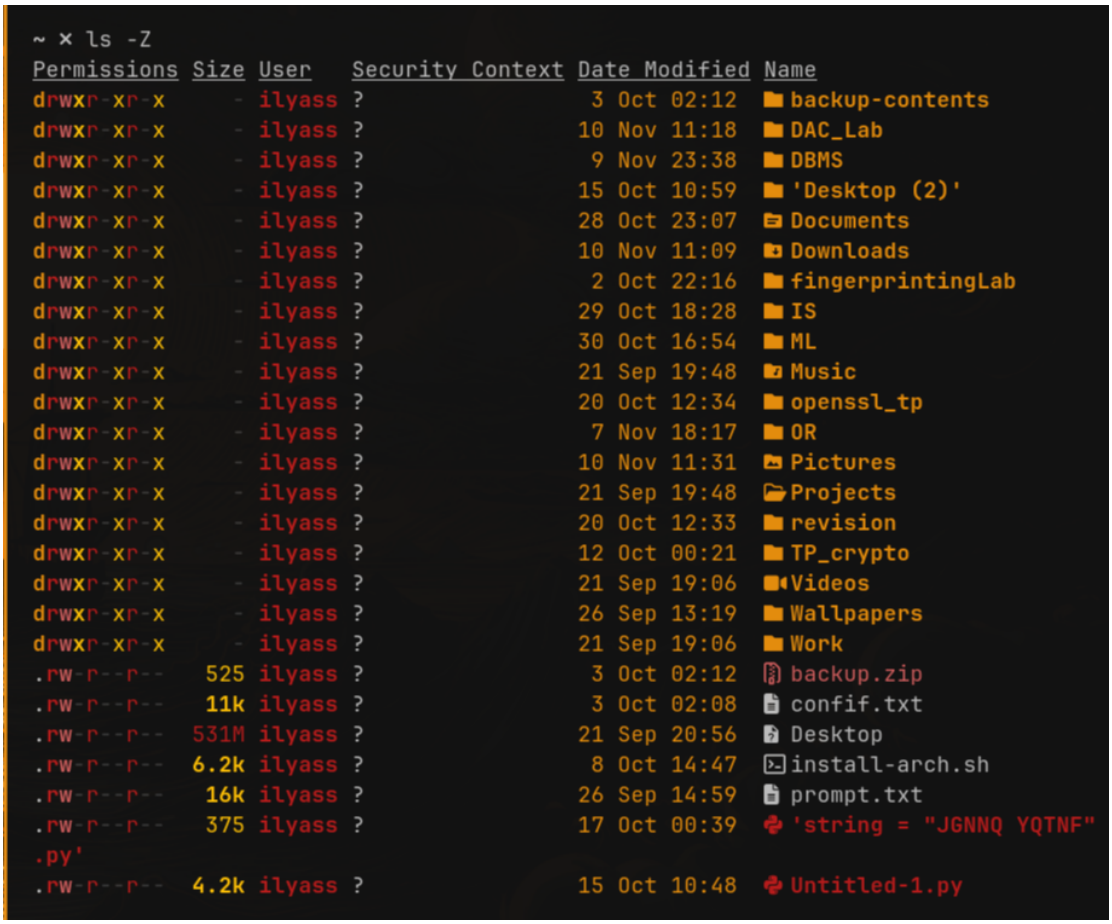


Figure 4: MAC exploration: `ls -Z` in the home directory; security contexts appear as `?`, with no `sestatus` evidence provided.

3.2 Challenge 2: Military Information System

Classification order: Public < Confidential < Secret. **Task 1: read-access matrix** (classification policy only):

User / Clearance	Document A / Secret	Document B / Confidential	Document C / Public
Alice / Secret	Allow	Allow	Allow
Bob / Confidential	Deny	Allow	Allow
Charlie / Public	Deny	Deny	Allow

Task 2: Bob reads Document A. Answer. Denied. Bob has Confidential clearance, whereas A is Secret. Granting the request would violate *no read up*.

Task 3: Alice writes to Document C. Answer. Denied. Alice has Secret clearance and C is Public. Writing downward violates *no write down* and could disclose secret data to lower-clearance readers.

Task 4: information leakage. Bell–LaPadula blocks reading above one's clearance and writing below one's clearance. These restrictions constrain information flow and help prevent classified data from reaching lower security levels through permitted read/write operations.

Task 5: MAC versus DAC. Answer. Sufficient clearance does not necessarily grant access. MAC clearance establishes that the mandatory classification rule permits an action, while DAC owner/group/other permissions may still deny it. When both mechanisms are enforced, both must

allow the request. DAC is discretionary (the owner manages permissions); MAC follows a system-enforced policy not freely overridable by the file owner.

4 RBAC: Role-Based Access Control

4.1 Guided experiment: users, roles, and directories

The roles from the handout are Administrator (full access), Developer (read/write where authorized), and Viewer (read-only where authorized). Groups can represent those roles.

The screenshot documents successful creation of `administrators`, `developers`, and `viewers`, followed by unsuccessful `adduser` attempts. On this Arch Linux installation, the correct alternative was `useradd -m -s /bin/bash`. Alice, Bob, and Charlie were then added to developers, viewers, and administrators respectively. The `groups` output confirms the memberships. The directories `/company/source` and `/company/reports` were created.

```
~ ❱ sudo groupadd administrators
sudo groupadd developers
sudo groupadd viewers
[sudo] password for ilyass:

~ ❱ sudo adduser alice
sudo adduser bob
sudo adduser charlie
sudo: adduser: command not found
sudo: adduser: command not found
sudo: adduser: command not found

~ ✕ sudo adduser alic
sudo: adduser: command not found

~ ✕ sudo adduser alice
sudo: adduser: command not found

~ ✕ sudo useradd -m -s /bin/bash alice

~ ❱ sudo useradd -m -s /bin/bash bob

~ ❱ sudo useradd -m -s /bin/bash charlie

~ ❱ sudo usermod -aG developers alice
sudo usermod -aG viewers bob
sudo usermod -aG administrators charlie

~ ❱ groups alice
groups bob
groups charlie
alice : alice developers
bob : bob viewers
charlie : charlie administrators

~ ❱ sudo mkdir -p /company/source
sudo mkdir -p /company/reports
```

Figure 5: RBAC setup: create role groups and users, replace unavailable `adduser` with `useradd`, check group memberships, and create organizational directories.

4.2 Challenge 3: Enterprise Access Management

Task 1: groups and users. The observed group assignments are Alice → developers, Bob → viewers, and Charlie → administrators.

Task 2: target permissions.

Role	Source code	Reports
Developer	Read and write	Read
Viewer	No access	Read
Administrator	Full access	Full access

Task 3: does the initial configuration satisfy the entire policy? Answer. No. Standard permissions with one owning group cannot independently grant all three roles their different permissions. In particular, developers need read access to reports in addition to viewers, and administrators need full access to both directories. The user attempted to solve this using ACLs.

The screenshot shows an ACL command granting administrators `rwx` on `/company/source`:

```
sudo setfacl -m g:administrators:rwx /company/source
```

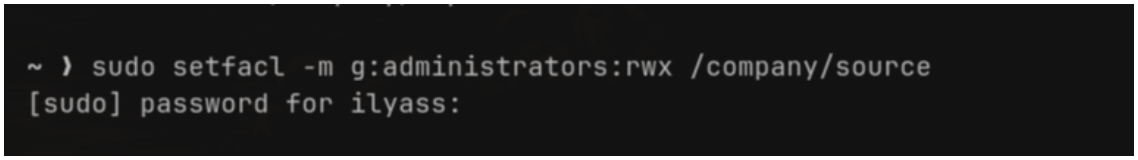


Figure 6: Challenge 3: granting the administrators group an additional ACL entry on the source directory.

A subsequent screenshot shows `chmod 744 /company/reports` and an administrator ACL entry. The `744` choice is insufficient for a shared directory: the group and others lack traversal permission `x`. It also does not grant the developer group the required report access. The ACL command itself has no visible error, but no `getfacl` verification is shown.

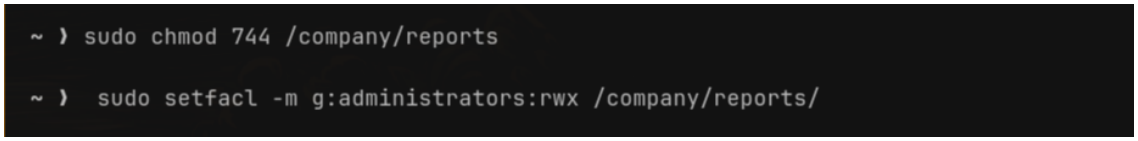


Figure 7: Challenge 3: reports directory set to mode 744 and administrator ACL added; the policy remains incomplete.

Task 4: corrected complete configuration. A suitable set of commands is:

```
# Source: developers rwx, administrators rwx, viewers none
sudo chown root:developers /company/source
sudo chmod 770 /company/source
sudo setfacl -m g:administrators:rwx /company/source

# Reports: viewers rx, developers rx, administrators rwx
sudo chown root:viewers /company/reports
sudo chmod 750 /company/reports
sudo setfacl -m g:developers:rx,g:administrators:rwx /company/reports

# Verify actual ACL entries and effective masks
getfacl /company/source
```

```
getfacl /company/reports
```

For directories, `r-x` enables listing and traversal (read-only directory access), and `rwx` permits managing directory entries. This proposed correction is *not* confirmed by the supplied screenshots. Additional ACLs or carefully planned ownership/default ACLs are needed if the policy must also cover existing and newly created files within the directories. For full end-to-end policy verification, test with each user's account and inspect both directory and file permissions.

5 ABAC: Attribute-Based Access Control

5.1 Challenge 4: Banking Access Policy

Policy from the lab: grant access only if the employee's department matches the document's department, the employee has adequate clearance, and the request occurs between 08:00 and 18:00 inclusive.

```
ALLOW if:
  user.department == document.department
  AND user.clearance >= document.classification
  AND 08:00 <= request.time <= 18:00
Otherwise DENY.
```

Tasks 1–2: evaluate requests and explain denials.

User	Time	Decision	Reason
Alice	10:00	Grant	Finance matches, Secret clearance sufficient, within hours
Bob	11:00	Deny	HR department does not match Finance document
Charlie	14:00	Deny	Public clearance insufficient for Secret Finance document
David	22:00	Deny	Outside allowed time window

Task 3: Alice requests an HR document. Answer. Denied under the given scenario, because Alice belongs to Finance, not HR. Even adequate clearance and a permitted time cannot override department mismatch.

Task 4: David requests at 17:00. Answer. Granted. Assuming his other attributes remain Finance/Secret and he requests the Finance/Secret document, 17:00 satisfies the time condition.

Task 5: why more flexible than RBAC? ABAC combines several changing attributes, such as department, document sensitivity and time of day, without requiring a separate static role for every possible combination. RBAC primarily grants rights through role membership. These are analytical answers to the handout; no ABAC implementation or screenshot was provided.

6 Summary and verification status

Model	Evidence provided	Assessment
DAC	File/directory creation, ownership commands, chmod and listing	Target modes 750/640 shown
MAC	`ls -Z` output	Theory answered; enforcement status unverified
RBAC	Group/user commands, `groups`, directories and ACL commands	Setup partly demonstrated; report permissions need correction and retesting
ABAC	Handout scenarios only	All five questions evaluated theoretically

Conclusion. DAC uses owner/group/other permission bits, MAC applies a mandatory classification policy, RBAC maps rights to organizational roles, and ABAC evaluates attributes and conditions. The lab demonstrates the limits of a single Linux owning group and shows why ACLs are needed for multi-role directory sharing. A final practical check with `getfacl` and actual user access attempts would provide stronger validation.